3420. Count Non-Decreasing Subarrays After K Operations

Daily Question - November 13, 2026

Problem Statement

You are given an array nums of n integers and an integer k.

For each subarray of nums, you can apply up to k operations. In each operation, you increment any element of the subarray by 1. Each subarray is considered independently, so changes made to one subarray do not persist to another.

Return the number of subarrays that can be made non-decreasing after performing at most k operations. An array is non-decreasing if each element is greater than or equal to its previous element, if it exists.

Examples

Example 1: nums = [6,3,1,2,4,4], k = 7. Output: 17. There are 21 total subarrays; exactly four subarrays beginning with [6,3,1] cannot be made non-decreasing using at most 7 operations.

Example 2: nums = [6,3,1,3,6], k = 4. Output: 12. All subarrays of length 1 or 2 are valid, [3,1,3,6] is valid, and [6,3,1] is invalid.

Constraints

1 <= nums.length <= 100000; 1 <= nums[i] <= 1000000000; 1 <= k <= 1000000000.

Solution: Sliding Window + Monotonic Deque

Only increments are allowed. Therefore, the cheapest way to make a subarray non-decreasing is to raise every element to the running maximum from the left. For a subarray [left, right], the minimum required increments are the sum of runningMax[i] - nums[i] for all i in the subarray.

Process left from right to left. For each left, find the largest right such that the required cost is at most k. If [left, right] is valid, every shorter subarray starting at left is also valid, so add right - left + 1 to the answer.

Maintain a monotonic deque of indices whose nums values are prefix maxima, in decreasing order. When adding a new left value, remove smaller maxima from the back and add the cost needed to raise their covered ranges to the new maximum. When the cost exceeds k, remove the rightmost element and update the cost using the oldest prefix maximum.

Complete Java Solution

```
import java.util.ArrayDeque;
import java.util.Deque;

class Solution {
    public long countNonDecreasingSubarrays(int[] nums, int k) {
        int n = nums.length;
        Deque<Integer> prefixMaxima = new ArrayDeque<>();
```

```
        long validSubarrays = 0;
        long requiredIncrements = 0;

        int right = n - 1;

        for (int left = n - 1; left >= 0; left--) {
            int currentValue = nums[left];

            while (!prefixMaxima.isEmpty()
                    && nums[prefixMaxima.peekLast()] < currentValue) {

                int removedIndex = prefixMaxima.pollLast();
                int nextIndex = prefixMaxima.isEmpty()
                        ? right + 1
                        : prefixMaxima.peekLast();

                int coveredCount = nextIndex - removedIndex;
                requiredIncrements +=
                        (long) coveredCount
                        * (currentValue - nums[removedIndex]);
            }

            prefixMaxima.addLast(left);

            while (requiredIncrements > k) {
                int oldestMaximum = prefixMaxima.peekFirst();

                requiredIncrements -=
                        nums[oldestMaximum] - nums[right];

                if (oldestMaximum == right) {
                    prefixMaxima.pollFirst();
                }

                right--;
            }

            validSubarrays += right - left + 1;
        }

        return validSubarrays;
    }
}
```

Correctness

For a subarray, the pointwise smallest non-decreasing array that is at least as large as the original at every position is its prefix maximum. Since only increments are allowed, the minimum number of operations is exactly the sum of differences between each prefix maximum and the original value.

For a fixed left endpoint, if the cost of [left, right] is at most k, then removing elements from the right cannot increase the cost. Hence all subarrays [left, r] with left <= r <= right are valid, and there are right - left + 1 of them.

The deque stores the prefix maxima in decreasing value order. When a new left value replaces smaller maxima, the cost increase is (new maximum - old maximum) times the number of positions previously covered by the removed maximum. When the cost exceeds k, removing the rightmost element decreases the cost by the difference between the oldest applicable maximum and the removed value. Thus the invariant is maintained after each adjustment.

Complexity

Time: O(n). Each index is added to and removed from the deque at most once, and both window pointers move monotonically. Auxiliary space: O(n) for the deque.

Implementation Notes

Use long for requiredIncrements and validSubarrays because values, k, and the number of subarrays can make intermediate totals exceed the 32-bit integer range. The deque stores indices, not values, so the cost can be updated correctly when the window shrinks.